Running the Entire Suite

Over the lifetime of a BDD project, your specs will run often, perhaps even thousands or tens of thousands of times. The time it takes to run the suite of specs will be multiplied by that huge number.

Consider the difference between a test suite taking 12 seconds and one taking 10 minutes. After 1,000 runs, the former has taken 3 hours and 20 minutes. The latter has cumulatively taken nearly 7 days. Team velocity takes a nosedive when they have to wait forever to find out if their changes broke anything. Throughout this book, we'll show you practices for keeping your specs snappy.

Getting Feedback from a Single Example

There's a huge difference between waiting less than a second for an example to run and waiting for several seconds or even multiple minutes. It's not just a quantitative shift. Once you've seen specs that give you near-instant feedback as you type, anything slower will feel like an unbearable interruption to your train of thought.

Gary Bernhardt makes effective use of this rapid-feedback development style, which he demonstrates in his *Destroy All Software* screencasts and blog posts:

> These tests are fast enough that I can hit enter (my test-running keystroke) and have a response before I have time to think. It means that the flow of my thoughts never breaks.[2]

Dealing with Failure

Failure is a good thing—really, it is. A failing spec points to the behavior that your recent change broke. However, it does cost time and energy to track down the source of the failure.

Much worse are failures caused by *brittle specs*—that is, specs that fail (perhaps intermittently) when the code is actually working. By writing precise RSpec expectations that describe exactly the behavior you're looking for—and no more—you keep your specs from becoming brittle.

Don't Overdo It!

Historically, developers have complained that their projects have lacked sufficient test suites. Now that TDD has become more mainstream, we're witnessing the opposite problem: projects that suffer from *overtesting*.

2. https://www.destroyallsoftware.com/blog/2014/tdd-straw-men-and-rhetoric